

**UNIVERSIDADE VEIGA DE ALMEIDA - UVA**

**Projeto e Análise de Algoritmos**

Brenno de Carvalho Prado Frangella Sciammarella

**Rio de Janeiro  
2026.1**

## Q1 - Problema do Troco

O Problema do Troco consiste em formar um determinado valor usando a menor quantidade possível de moedas.

Foram implementadas duas abordagens:

- Algoritmo guloso.
- Programação dinâmica.

### Algoritmo guloso

O algoritmo guloso ordena as moedas em ordem decrescente e sempre escolhe a maior moeda possível para o valor restante.

Passos:

- Ordenar as moedas da maior para a menor.
- Pegar a maior quantidade possível da moeda atual.
- Subtrair o valor usado do troco restante.
- Repetir o processo para as próximas moedas.

Complexidade:

- Tempo:  $O(M \log M)$  por causa da ordenação, mais  $O(M)$  para percorrer as moedas.
- Espaço:  $O(M)$  para armazenar a composição da resposta.

A vantagem é ser rápido e simples. A desvantagem é que nem sempre encontra a menor quantidade possível de moedas.

### Programação dinâmica

A programação dinâmica calcula a melhor resposta para todos os valores de 0 até o valor do troco. O vetor  $dp[v]$  guarda a menor quantidade de moedas necessária para formar o valor  $v$ .

Passos:

- Criar o vetor  $dp$ .
- Definir  $dp[0] = 0$ .
- Para cada valor, testar todas as moedas.
- Guardar a menor quantidade encontrada.
- Reconstruir a composição final da resposta.

Complexidade:

- Tempo:  $O(M * V)$ , em que  $M$  é a quantidade de moedas e  $V$  é o valor do troco.
- Espaço:  $O(V + M)$ .

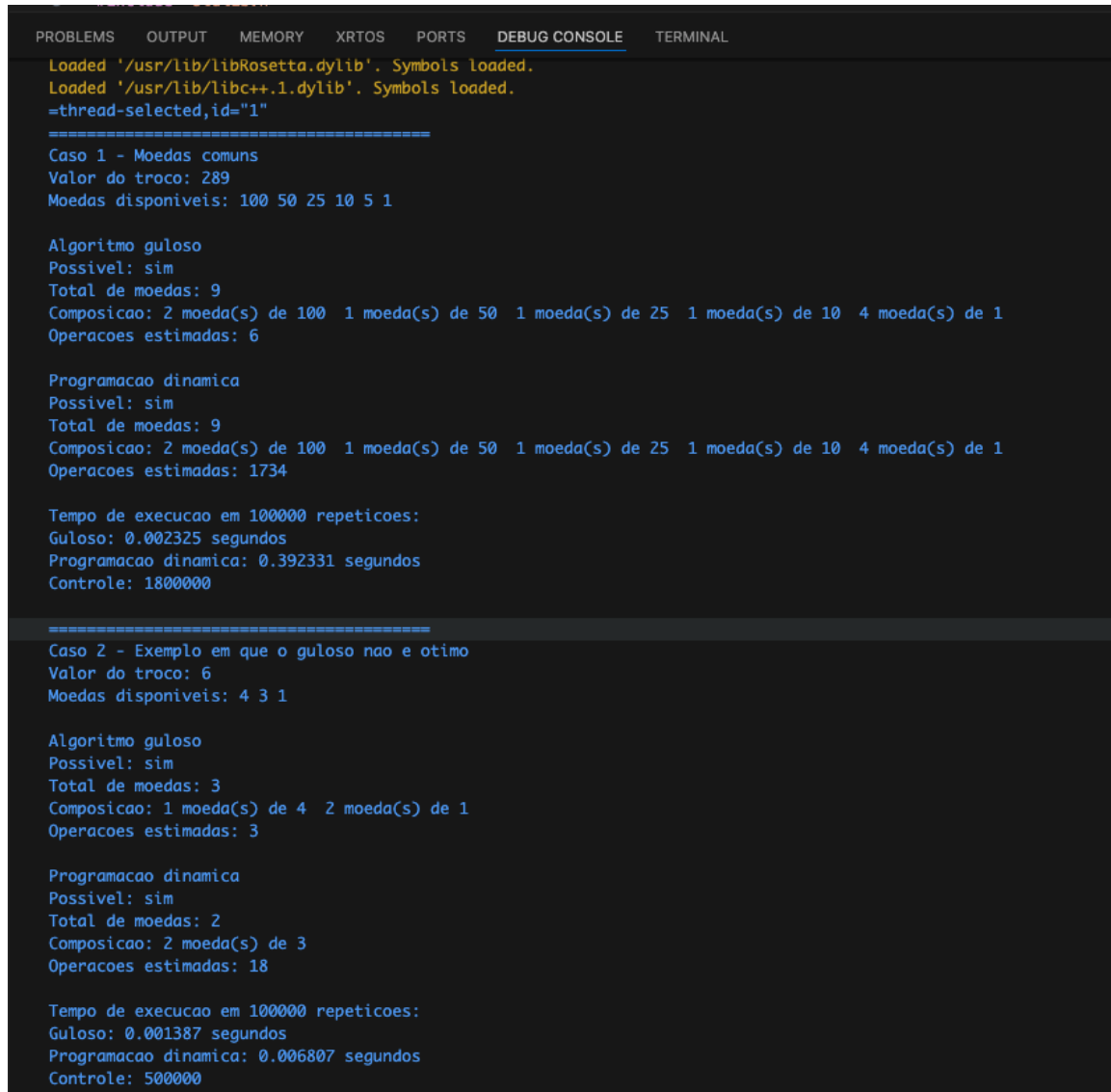
A vantagem é sempre encontrar a melhor resposta quando o troco pode ser formado. A desvantagem é ter custo maior que o algoritmo guloso.

### Comparação

No primeiro caso do código, usando moedas comuns, os dois algoritmos encontram a mesma resposta. No segundo caso, usando moedas (1, 3, 4) para formar o valor 6, o guloso escolhe  $4 + 1 + 1$ , totalizando 3 moedas. A programação dinâmica encontra  $3 + 3$ , totalizando 2 moedas.

Tambem foi incluida uma medicao simples de tempo usando `clock()`. Cada algoritmo e executado varias vezes para que a diferenca fique visivel. Os valores em segundos podem mudar conforme o computador usado, mas a tendencia esperada e que o algoritmo guloso seja mais rapido, enquanto a programacao dinamica faca mais trabalho para garantir a resposta otima.

## Execução da Q1



```
Loaded '/usr/lib/libRosetta.dylib'. Symbols loaded.
Loaded '/usr/lib/libc++.1.dylib'. Symbols loaded.
=thread-selected,id="1"
=====
Caso 1 - Moedas comuns
Valor do troco: 289
Moedas disponiveis: 100 50 25 10 5 1

Algoritmo guloso
Possivel: sim
Total de moedas: 9
Composicao: 2 moeda(s) de 100 1 moeda(s) de 50 1 moeda(s) de 25 1 moeda(s) de 10 4 moeda(s) de 1
Operacoes estimadas: 6

Programacao dinamica
Possivel: sim
Total de moedas: 9
Composicao: 2 moeda(s) de 100 1 moeda(s) de 50 1 moeda(s) de 25 1 moeda(s) de 10 4 moeda(s) de 1
Operacoes estimadas: 1734

Tempo de execucao em 100000 repeticoes:
Guloso: 0.002325 segundos
Programacao dinamica: 0.392331 segundos
Controle: 180000

=====
Caso 2 - Exemplo em que o guloso nao e otimo
Valor do troco: 6
Moedas disponiveis: 4 3 1

Algoritmo guloso
Possivel: sim
Total de moedas: 3
Composicao: 1 moeda(s) de 4 2 moeda(s) de 1
Operacoes estimadas: 3

Programacao dinamica
Possivel: sim
Total de moedas: 2
Composicao: 2 moeda(s) de 3
Operacoes estimadas: 18

Tempo de execucao em 100000 repeticoes:
Guloso: 0.001387 segundos
Programacao dinamica: 0.006807 segundos
Controle: 500000
```

Figura 1 - Execucao do codigo Questao1.c, mostrando os dois casos de teste, a comparacao entre algoritmo guloso e programacao dinamica, e a diferenca no tempo de execucao.

## Codigo da Q1

```
#include <limits.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MAX_MOEDAS 20
#define MAX_VALOR 10000
#define REPETICOES_TEMPO 100000

typedef struct {
    int possivel;
    int total_moedas;
    int usadas[MAX_MOEDAS];
    long operacoes;
} Resultado;

int comparar_decrescente(const void *a, const void *b) {
    int x = *(const int *)a;
```

```

int y = *(const int *)b;

if (x < y) {
return 1;
}
if (x > y) {
return -1;
}
return 0;
}

void zerar_resultado(Resultado *r, int n) {
int i;

r->possivel = 0;
r->total_moedas = 0;
r->operacoes = 0;

for (i = 0; i < n; i++) {
r->usadas[i] = 0;
}
}

Resultado algoritmo_guloso(int moedas[], int n, int valor) {
Resultado r;
int restante;
int i;
int quantidade;

zerar_resultado(&r, n);

restante = valor;

for (i = 0; i < n; i++) {
quantidade = restante / moedas[i];
r.usadas[i] = quantidade;
r.total_moedas += quantidade;
restante -= quantidade * moedas[i];
r.operacoes++;
}

r.possivel = (restante == 0);
return r;
}

Resultado programacao_dinamica(int moedas[], int n, int valor) {
Resultado r;
int dp[MAX_VALOR + 1];
int ultima_moeda[MAX_VALOR + 1];
int i;
int v;
int atual;
int indice;

zerar_resultado(&r, n);

for (v = 0; v <= valor; v++) {
dp[v] = INT_MAX / 2;
ultima_moeda[v] = -1;
}

dp[0] = 0;

for (v = 1; v <= valor; v++) {
for (i = 0; i < n; i++) {
r.operacoes++;

if (moedas[i] <= v && dp[v - moedas[i]] + 1 < dp[v]) {
dp[v] = dp[v - moedas[i]] + 1;
ultima_moeda[v] = i;
}
}
}

if (dp[valor] < INT_MAX / 2) {
r.possivel = 1;
r.total_moedas = dp[valor];
atual = valor;

while (atual > 0) {
indice = ultima_moeda[atual];

if (indice < 0) {
break;
}

r.usadas[indice]++;
atual -= moedas[indice];
}
}

```

```

}

return r;
}

void imprimir_resultado(const char *nome, int moedas[], int n, Resultado r) {
int i;

printf("%s\n", nome);
printf("Possivel: %s\n", r.possivel ? "sim" : "nao");

if (r.possivel) {
printf("Total de moedas: %d\n", r.total_moedas);
printf("Composicao: ");

for (i = 0; i < n; i++) {
if (r.usadas[i] > 0) {
printf("%d moeda(s) de %d ", r.usadas[i], moedas[i]);
}
}

printf("\n");
}

printf("Operacoes estimadas: %ld\n\n", r.operacoes);
}

void medir_tempo(int moedas[], int n, int valor) {
clock_t inicio;
clock_t fim;
double tempo_guloso;
double tempo_dinamico;
int i;
int soma_controle;
Resultado r;

soma_controle = 0;

inicio = clock();
for (i = 0; i < REPETICOES_TEMPO; i++) {
r = algoritmo_guloso(moedas, n, valor);
soma_controle += r.total_moedas;
}
fim = clock();
tempo_guloso = (double)(fim - inicio) / CLOCKS_PER_SEC;

inicio = clock();
for (i = 0; i < REPETICOES_TEMPO; i++) {
r = programacao_dinamica(moedas, n, valor);
soma_controle += r.total_moedas;
}
fim = clock();
tempo_dinamico = (double)(fim - inicio) / CLOCKS_PER_SEC;

printf("Tempo de execucao em %d repeticoes:\n", REPETICOES_TEMPO);
printf("Guloso: %.6f segundos\n", tempo_guloso);
printf("Programacao dinamica: %.6f segundos\n", tempo_dinamico);
printf("Controle: %d\n\n", soma_controle);
}

void executar_caso(const char *titulo, int moedas_originais[], int n, int valor) {
int moedas[MAX_MOEDAS];
int i;
Resultado guloso;
Resultado dinamico;

for (i = 0; i < n; i++) {
moedas[i] = moedas_originais[i];
}

qsort(moedas, n, sizeof(int), comparar_decrescente);

printf("=====\n");
printf("%s\n", titulo);
printf("Valor do troco: %d\n", valor);
printf("Moedas disponiveis: ");

for (i = 0; i < n; i++) {
printf("%d ", moedas[i]);
}

printf("\n\n");

guloso = algoritmo_guloso(moedas, n, valor);
dinamico = programacao_dinamica(moedas, n, valor);

imprimir_resultado("Algoritmo guloso", moedas, n, guloso);
imprimir_resultado("Programacao dinamica", moedas, n, dinamico);
medir_tempo(moedas, n, valor);

```

```

}

int main(void) {
int moedas_caso_1[] = {1, 5, 10, 25, 50, 100};
int moedas_caso_2[] = {1, 3, 4};

executar_caso("Caso 1 - Moedas comuns", moedas_caso_1, 6, 289);
executar_caso("Caso 2 - Exemplo em que o guloso nao e otimo", moedas_caso_2, 3, 6);

return 0;
}

```

## Q2 - Analise de Complexidade Big O

A tabela abaixo apresenta a complexidade dos algoritmos do enunciado.

| Algoritmo     | Complexidade  | Justificativa                                                                                      |
|---------------|---------------|----------------------------------------------------------------------------------------------------|
| Algoritmo1(n) | $O(\log n)$   | A variavel i comeca em n e e dividida por 2 a cada repeticao.                                      |
| Funcao2(n)    | $O(\log n)$   | A funcao faz apenas uma chamada recursiva para $n / 2$ .                                           |
| Fib(n)        | $O(2^n)$      | Cada chamada gera duas novas chamadas, repetindo muitos subproblemas.                              |
| Funcao4(n)    | $O(n \log n)$ | A recorrência é $T(n) = T(n/3) + T(2n/3) + O(n)$ . Cada nível soma $O(n)$ e ha altura logaritmica. |

### Algoritmo1(n)

```

cont = 0
i = n
enquanto i > 1 faca:
    i = i / 2
    cont = cont + 1

```

A cada repeticao, i e dividido por 2. Depois de k repeticoes, o valor sera aproximadamente  $n / 2^k$ . O laço termina quando esse valor fica menor ou igual a 1. Portanto, a complexidade é  $O(\log n)$ .

### Funcao2(n)

```

Funcao2(n):
se n <= 1 entao:
    retornar 1
senao:
    retornar Funcao2(n / 2) + 1

```

A funcao reduz o problema pela metade a cada chamada e faz apenas uma chamada recursiva. A recorrência é  $T(n) = T(n/2) + O(1)$ . Portanto, a complexidade é  $O(\log n)$ .

### Fib(n)

```

Fib(n):
se n <= 1 entao:
    retornar n
senao:
    retornar Fib(n - 1) + Fib(n - 2)

```

Essa implementacao recursiva recalcula os mesmos valores varias vezes. Como cada chamada pode gerar duas novas chamadas, a quantidade de chamadas cresce exponencialmente. Portanto, a complexidade é  $O(2^n)$ .

### Funcao4(n)

```

Funcao4(n):
se n <= 1 entao:
    retornar
senao:
    Funcao4(n / 3)
    Funcao4(2n / 3)
    para i de 1 ate n faca:
        operacao_constante()

```

A funcao divide o problema em duas partes e ainda executa um laco linear. A soma dos tamanhos dos subproblemas em cada nivel e  $n$ , e a altura da arvore de recursao e logaritmica. Portanto, a complexidade e  $O(n \log n)$ .

### Q3 - Sistema de Autocompletar com Restricoes

A terceira questao foi implementada usando uma trie. Uma trie e uma arvore em que cada caminho representa um prefixo. Quando uma palavra proibida termina, o no correspondente e marcado como fim de palavra.

O codigo usa palavras proibidas fixas no main: spam, golpe e promo. Depois, ele adiciona a palavra bot para demonstrar a operacao ADD.

#### Operacoes implementadas

- ADD: adiciona uma nova palavra proibida na trie.
- CHECK: verifica se o texto comeca com alguma palavra proibida.
- COUNT: conta quantas palavras proibidas possuem determinado prefixo.

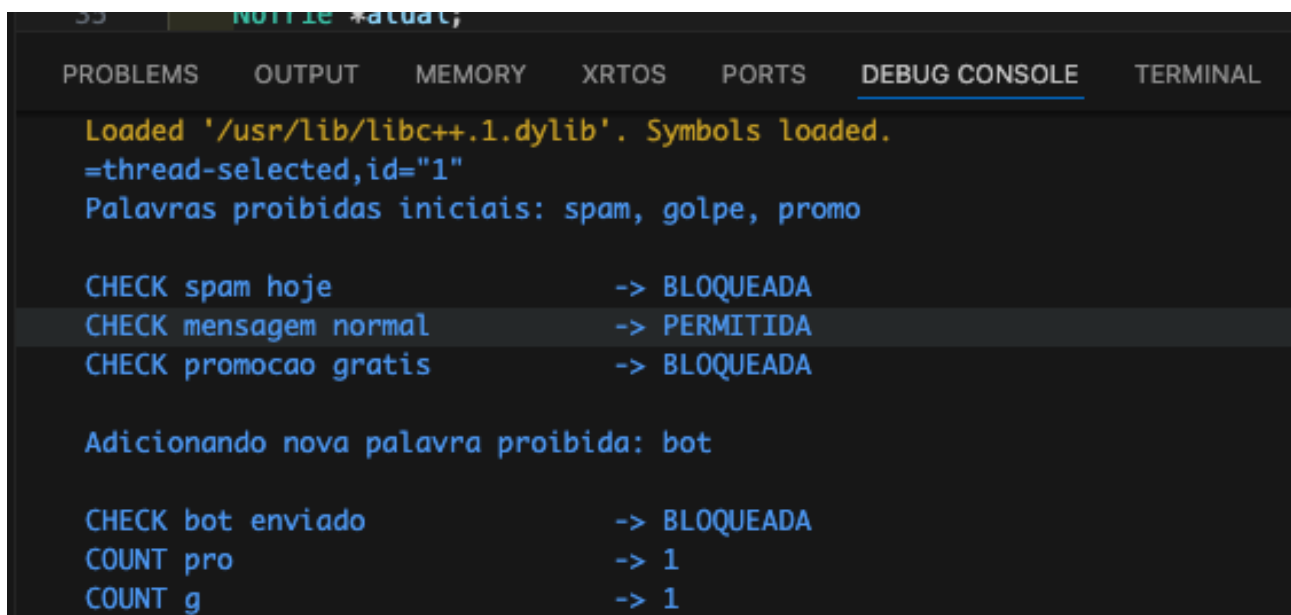
#### Funcionamento

Para verificar um texto, o programa percorre seus caracteres na trie. Se durante esse percurso encontrar um no marcado como fim de palavra, significa que o texto comeca com uma palavra proibida e deve ser bloqueado.

Complexidade:

- Insercao:  $O(L)$ , em que  $L$  e o tamanho da palavra.
- CHECK:  $O(T)$ , em que  $T$  e o tamanho do texto consultado.
- COUNT:  $O(P)$ , em que  $P$  e o tamanho do prefixo.
- Espaco:  $O(S)$ , em que  $S$  e a soma dos tamanhos das palavras armazenadas.

#### Execução da Q3



```
33  NOTIE *actual;

PROBLEMS  OUTPUT  MEMORY  XRTOS  PORTS  DEBUG CONSOLE  TERMINAL

Loaded '/usr/lib/libc++.1.dylib'. Symbols loaded.
=thread-selected,id="1"
Palavras proibidas iniciais: spam, golpe, promo

CHECK spam hoje          -> BLOQUEADA
CHECK mensagem normal    -> PERMITIDA
CHECK promocao gratis     -> BLOQUEADA

Adicionando nova palavra proibida: bot

CHECK bot enviado         -> BLOQUEADA
COUNT pro               -> 1
COUNT g                 -> 1
```

Figura 2 - Execucao do codigo Questao3.c, mostrando as palavras proibidas iniciais, as verificacoes com CHECK, a inclusao de uma nova palavra com ADD e as consultas de prefixo com COUNT.

#### Codigo da Q3

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define ALFABETO 256

typedef struct NoTrie {
    struct NoTrie *filhos[ALFABETO];
    int fim_de_palavra;
    int qtd_prefixo;
} NoTrie;

NoTrie *criar_no(void) {
    NoTrie *no;
    int i;

    no = (NoTrie *)malloc(sizeof(NoTrie));

    if (no == NULL) {
        printf("Erro ao alocar memoria.\n");
        exit(1);
    }

    no->fim_de_palavra = 0;
    no->qtd_prefixo = 0;

    for (i = 0; i < ALFABETO; i++) {
        no->filhos[i] = NULL;
    }

    return no;
}

int contem_palavra(NoTrie *raiz, const char *palavra) {
    NoTrie *atual;
    unsigned char c;
    int i;

    atual = raiz;

    for (i = 0; palavra[i] != '\0'; i++) {
        c = (unsigned char)palavra[i];

        if (atual->filhos[c] == NULL) {
            return 0;
        }

        atual = atual->filhos[c];
    }

    return atual->fim_de_palavra;
}

void inserir(NoTrie *raiz, const char *palavra) {
    NoTrie *atual;
    unsigned char c;
    int i;

    if (palavra[0] == '\0' || contem_palavra(raiz, palavra)) {
        return;
    }

    atual = raiz;
    raiz->qtd_prefixo++;

    for (i = 0; palavra[i] != '\0'; i++) {
        c = (unsigned char)palavra[i];

        if (atual->filhos[c] == NULL) {
            atual->filhos[c] = criar_no();
        }

        atual = atual->filhos[c];
        atual->qtd_prefixo++;
    }

    atual->fim_de_palavra = 1;
}

int texto_bloqueado(NoTrie *raiz, const char *texto) {
    NoTrie *atual;
    unsigned char c;
    int i;

    atual = raiz;

    for (i = 0; texto[i] != '\0'; i++) {
        c = (unsigned char)texto[i];
    }

```

```

if (atual->filhos[c] == NULL) {
return 0;
}

atual = atual->filhos[c];

if (atual->fim_de_palavra) {

return 1;
}
}

return 0;
}

int contar_com_prefixo(NoTrie *raiz, const char *prefixo) {
NoTrie *atual;
unsigned char c;
int i;

atual = raiz;

for (i = 0; prefixo[i] != '\0'; i++) {
c = (unsigned char)prefixo[i];

if (atual->filhos[c] == NULL) {
return 0;
}

atual = atual->filhos[c];
}

return atual->qtd_prefixo;
}

void liberar_trie(NoTrie *no) {
int i;

if (no == NULL) {
return;
}

for (i = 0; i < ALFABETO; i++) {
liberar_trie(no->filhos[i]);
}

free(no);
}

void consultar_texto(NoTrie *raiz, const char *texto) {
printf("CHECK %-25s -> %s\n", texto, texto_bloqueado(raiz, texto) ? "BLOQUEADA" : "PERMITIDA");
}

void consultar_prefixo(NoTrie *raiz, const char *prefixo) {
printf("COUNT %-25s -> %d\n", prefixo, contar_com_prefixo(raiz, prefixo));
}

int main(void) {
NoTrie *raiz;

raiz = criar_no();

inserir(raiz, "spam");
inserir(raiz, "golpe");
inserir(raiz, "promo");

printf("Palavras proibidas iniciais: spam, golpe, promo\n\n");

consultar_texto(raiz, "spam hoje");
consultar_texto(raiz, "mensagem normal");
consultar_texto(raiz, "promocao gratis");

printf("\nAdicionando nova palavra proibida: bot\n\n");
inserir(raiz, "bot");

consultar_texto(raiz, "bot enviado");
consultar_prefixo(raiz, "pro");
consultar_prefixo(raiz, "g");

liberar_trie(raiz);

return 0;
}

```

## Repositorio do trabalho

[https://github.com/SciammarellaB/Projeto\\_Analise\\_Algoritmo](https://github.com/SciammarellaB/Projeto_Analise_Algoritmo)